2. Beadandó dokumentáció

BHYUHA – Sándor Kolos Barnabás

1.fejezet

Feladat:

Készítsünk programot, amellyel egy Rubik órát lehet kirakni. A játékban 9 óra kap helyet, amely 1-12 közötti értéket mutatnak (kezdetben véletlenszerűen beállítva), és 3×3-as formában jelennek a játéktáblán. Az órák között az átlóknál 4 gomb helyezkedik el, amelyek a szomszédos 4 óra állását tudják eggyel növelni (tehát 4 óra van, amit csak egy gomb növel, 4, amit kettő, és 1, amit mind a négy gomb növel). A kezdő állásban az órák véletlenszerű időt mutatnak. A játék célja az, hogy a gombokkal történő állítással mind a 9 óra 12-t mutasson. A program biztosítson lehetőséget új játék kezdésére, és ismerje fel, ha vége a játéknak. Ekkor jelenítse meg, hány lépéssel (állítással) győzött a játékos, majd kezdjen automatikusan új játékot.

2.fejezet

Rövid elemzés

Az előző beadandóval szemben – valamivel izgalmasabb módon – itt nem egy szimulációt, hanem egy valódi játékot kell majd elkészíteni. Ez azt jelenti, hogy nagyobb a kreatív tér, azonban itt még jobban résen kell lenni, hogy a felhasználó hogyan tudja elrontani a program működését.

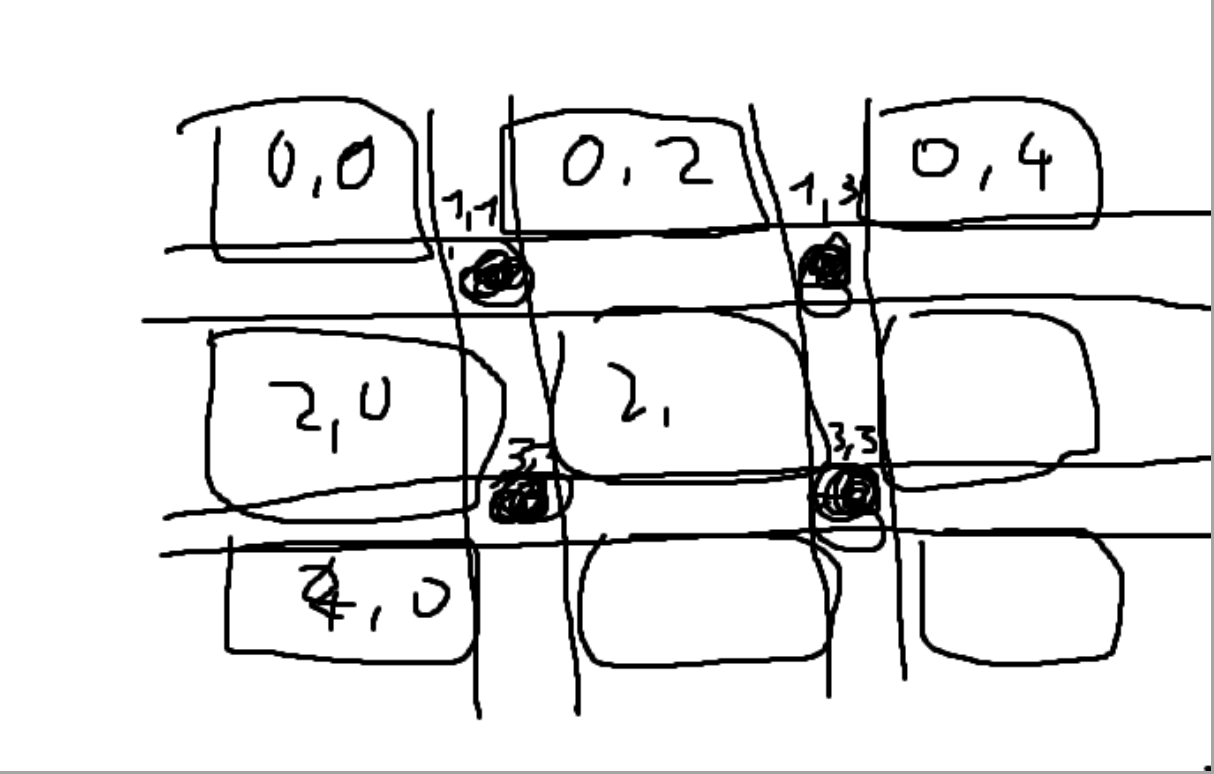
A játék maga azonban nem bonyolult és a „játéktéren” sem történik sok interakció, tehát ettől függetlenül a feladat még bőven megoldható.

Megoldás terve

Mivel a feladat nem összetett a package-ek struktúráját sem kell túlkomplikálni, úgyhogy a logikai munka legnagyobb része egy .java fájlba lesz gyűjtve.

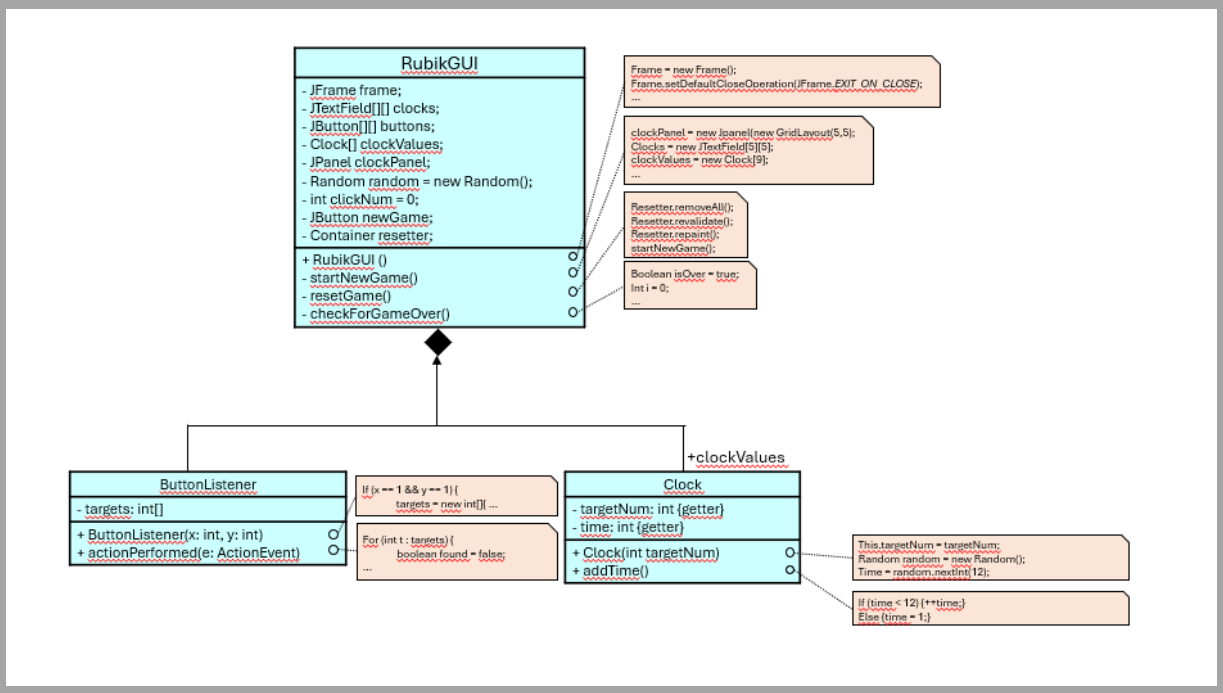
Az első hely ahol elakadtam mindjárt az elején volt, miszerint hogyan tegyem az átlókba a gombokat, mégis milyen struktúrája legyen a játéknak. Két lehetséges utat láttam: az egyik, hogy két szintes lesz a játék felépítése és a gombok az órák egy táblázatán fognak elhelyezkedni, a másik, amit végül választottam, hogy egy nagy táblázatban helyezkedik el minden. Ezen dilemma után szilárd tempóban haladt tovább a fejlesztés, egyedül az optimalizációkkal és kényelmi javításokkal kellett foglalkozni huzamosabb ideig, de a játék működőképes volt.

(korai tervrajz a játék kinézetéről):



3.fejezet

Megvalósítás:



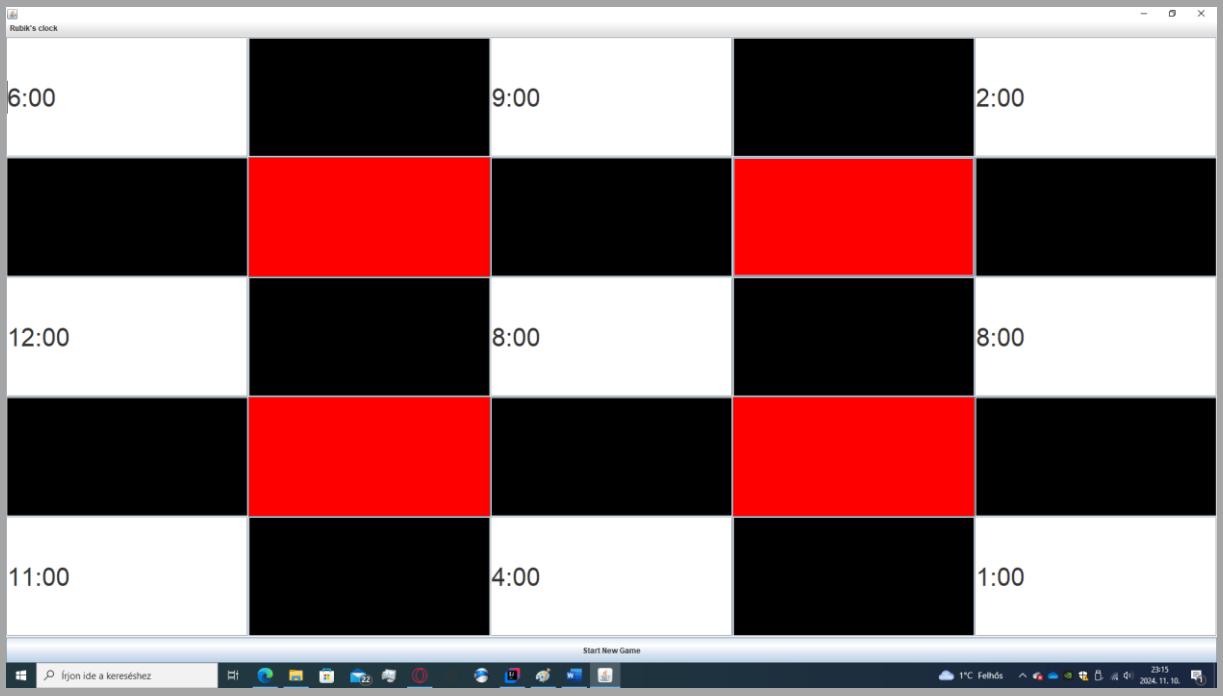
kifejezett különlegesebb függvény nincs ebben a programban, de csak hogy párat megemlítsek:

-RubikGUI() és startNewGame(): gondolkoztam hogy lehetne legrövidebben megoldani egy új játék kezdetét, azonban röviden akartam megcsinálni, nem szerettem volna még 40 sort írni, ekkor jött az ötlet, hogy csak átmásolom egy specifikus részét a konstruktornak, majd abban – és ahol még kell – meghívom a készített függvényt

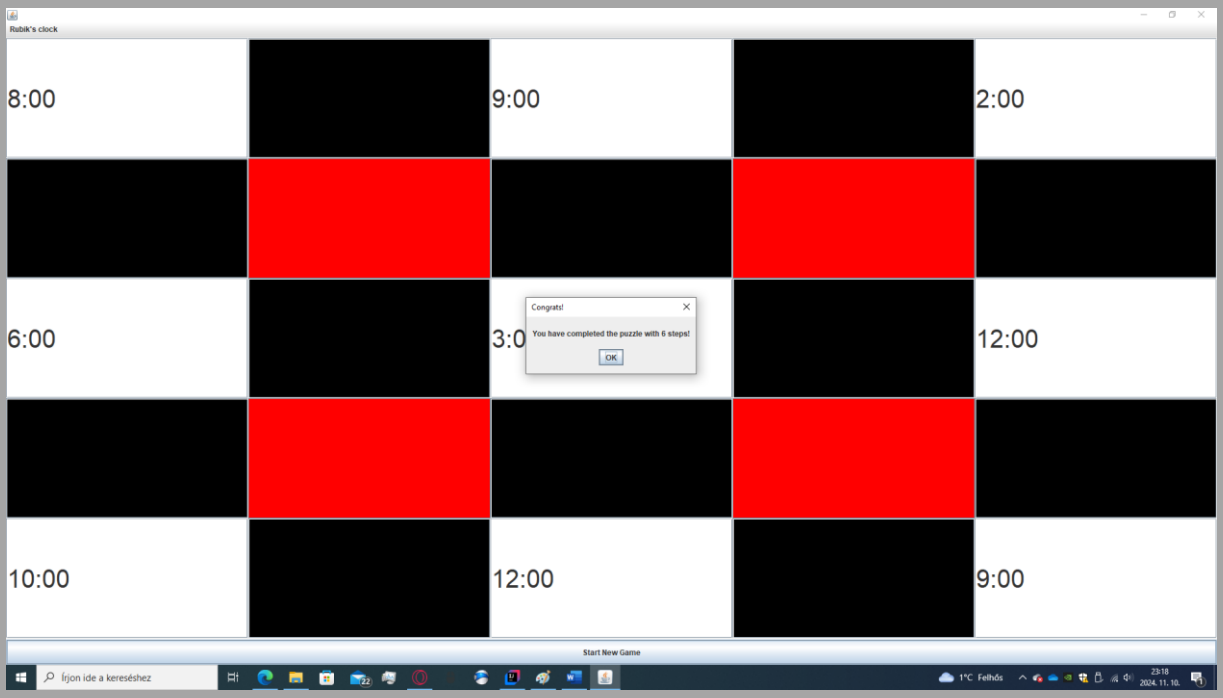
-Clock() (illetve a Clock osztály): itt sincs nagy megoldás, azonban számomra pont a rövidsége miatt tűnik ki. Először egy hosszú és csúnya válasszal hozakodtam elő a problémára, de végül rájöttem a hibára és ki is javítottam :)

-addTime(): az egyetlen függvény amit nem kellett megváltoztatnom és elsőre jó lett :D

képernyőkép futás közben:



képernyőkép a játék végének tesztelésekor:



4.fejezet

shorts: okos buta programozó(lusta, instant becsukja)

As a	okos játékos	
I want to	megoldani a feladványt minél gyorsabban	
1.	GIVEN	elindítjuk a játékot
	WHEN	megoldja helyesen a feladványt 6 lépésből
	THEN	megnyeri a játékot, ez könnyű volt
2.	GIVEN	elindítjuk a játékot
	WHEN	sajnos mindegyik 12:00-n kezdett
	THEN	innentől nem lesz sok dolga a játékosnak
3.	GIVEN	elindítjuk a játékot
	WHEN	megoldja helyesen a feladványt 42 lépésből
	THEN	megnyeri a játékot, ez már nehezebb volt
4.	GIVEN	elindítjuk a játékot
	WHEN	nehezen birkózik egy feladvánnyal, inkább indít egy új játékot
	THEN	elindul a játék egy újabb kiosztással
5.	GIVEN	elindítjuk a játékot
	WHEN	elege lesz a feladványból, inkább azonnal kilövi az ablakot
	THEN	bezáródik az ablak
As a	buta játékos	
I want to	megoldani a feladványt (lehetőleg)	
6.	GIVEN	elindítjuk a játékot
	WHEN	megoldja helyesen a feladványt 325 lépésből
	THEN	megnyeri a játékot, ez meg sem kottyant
7.	GIVEN	elindítjuk a játékot
	WHEN	mindegyik 12:00-n kezdett, szerencse!!!
	THEN	innentől könnyű dolga lesz a játékosnak (elrontotta)
8.	GIVEN	elindítjuk a játékot
	WHEN	1242 lépésnél tart, inkább kilép
	THEN	bezáródik az ablak
9.	GIVEN	elindítjuk a játékot
	WHEN	nehezen birkózik egy feladvánnyal, inkább indít egy új játékot
	THEN	elindul a játék egy újabb kiosztással

10.	GIVEN	elindítjuk a játékot
	WHEN	elege lesz a feladványból, inkább azonnal kilövi az ablakot
	THEN	rossz gombot nyomott meg, új játék kezdődik
As a	programozó	
I want to	hatékony lenni	
11.	GIVEN	elindítjuk a játékot
	WHEN	mivel látja, hogy a feladvány nem oldható meg 6 lépés alatt inkább hagyja és új játékot indít (nem szereti a brute forceot)
	THEN	elindul a játék egy újabb kiosztással
12.	GIVEN	elindítjuk a játékot
	WHEN	szerencsés a kiosztás, 4 lépésből megoldja a játékot
	THEN	megnyeri a játékot, ez könnyű volt
13.	GIVEN	elindítjuk a játékot
	WHEN	megpróbál a textfieldekbe írni, hogy megtörje a játékot
	THEN	nem sikerül, a textfieldben a következő gombnyomáskor már a helyes érték jelenik meg
14.	GIVEN	elindítjuk a játékot
	WHEN	előző próbálkozásból tanulva külső alkalmazással akarja megváltoztatni az órák értékeit
	THEN	sajnos helyes a változók láthatósága, nem fér hozzájuk
15.	GIVEN	el sem indítja a játékot
	WHEN	tudja, hogy úgysem cselezheti ki a készítőt
	THEN	a játék nem indul el